Utvecklingsmetodik

En rejäl crash course i pakethantering, testning och paketering

Dessutom: projektet!

Johan Holmberg 2026-05-04

Dagens föreläsning

- Pakethantering
- Paketering
- Testning

MALMÖ UNIVERSITET

Metodik

But why?

Mjukvaruutveckling är dyrt!

- Våra verktyg är dyra
- Våra miljöer är dyra
- Vi är dyra
- **Våra fel** är dyra

Felen vi gör resulterar i:

MALMÖ UNIVERSITET

Produktionsbortfall

MALMÖ UNIVERSITET

Tappad försäljning

Biljettkaoset: Kunder uppmanades åka gratis

SJ Publicerad 6 okt 2017 kl 08.57

Många försökte förgäves köpa biljetter på Stockholms centralstation på fredagen. Men vid lunchtid …

Foto: LEIF BRÄNNSTRÖM

MALMÖ UNIVERSITET

Olyckor

MALMÖ UNIVERSITET

Dödsfall

MALMÖ UNIVERSITET

Eller värre...

MALMÖ UNIVERSITET

Ja, men varför?

MALMÖ UNIVERSITET

MALMÖ UNIVERSITET

Also, AI...

13-hour AWS outage reportedly caused by Amazon's own AI tools

AWS outages caused by AI tools • Engadget / by Lawrence Bonk / Feb 20, 2026 at 10:49 PM

MALMÖ UNIVERSITET

Kodapan

Skriver kod. That’s it.

Ett kugghjul i ett maskineri:

- Skriver kod mekaniskt
- Saknar överblick
- Utan egentligt ansvar
- Väldigt utbytbar

Vibe-kodaren

Kopierar mög från ChatGPT et al.

En glorifierad produktägare:

- Blir bra på att outsourca utveckling till AI
- Saknar (ofta) överblick
- Väldigt utbytbar

Cowboy-kodaren

Skriver kod. That’s it.

En “fri själ”, men inte så bra:

- Ofta orutinerad
- Ostrukturerad
- Saknar kvalitetstänk
- Inte hållbar

Programmeraren

Skriver bra kod. Programmerar.

En renässansmänniska:

- Skapande
- Konstnärlig
- Ensamt geni
- Inte hållbar i längden

Utvecklaren

Skapar lösningar. Utvecklar.

En ingenjörstyp:

- Metodisk och strukturerad
- Grupporienterad
- Kvalitetstänkande
- Hållbar

Hur blir vi ordentliga utvecklare?

Genom att arbeta strukturerat och rationellt!

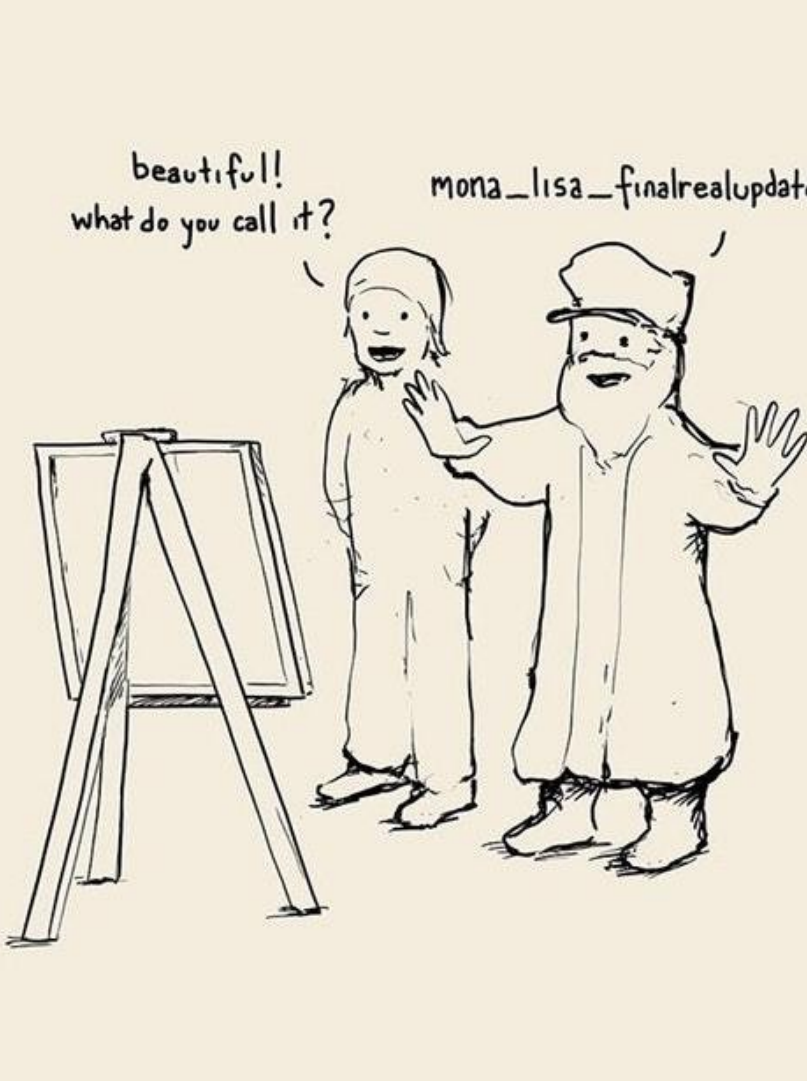
- Tänk först – skriv kod sedan. Gör vi rätt saker?
- Kan vi återanvända tidigare skriven kod? Hur?
- Kan vi effektivt samarbeta kring vår mjukvara?
- Hur dokumenterar vi?
- Hur säkerställer vi att koden fungerar?

MALMÖ UNIVERSITET

Versionshantering

Vårt problem

- Riktiga mjukvaruprojekt är stora
- Riktiga mjukvaruprojekt kommer ofta att innebära flera – och parallella – releaser av samma mjukvara
- Hur hanterar vi detta på ett effektivt sätt?



Vad är versionshantering?

Versionshantering är den metod med vilken vi kan hantera flera parallella versioner av en mjukvara

- Möjliggör samtidig utveckling av olika delar av mjukvaran
- Ger oss möjligheter att känna till – och dra nytta av – tidigare versioner av mjukvaran
- Möjliggör samtidig utveckling av nischade versioner eller utgåvor av en mjukvara

MALMÖ UNIVERSITET

Fördelarna med versionshantering

- EN SANNING – Vi kan alla vara överens om vilken kod som är den aktuella
- Samtidig utveckling
  - Gör det enklare att dela och återanvända kod
  - Olika team kan arbeta på samma projekt
- Historia och spårbarhet
  - Återgå till tidigare versioner av ett projekt
    - Förstörde din kursare din kod? Ingen fara, den finns kvar!
  - Underhåll flera olika releaser av samma fil, modul eller applikation
  - Vem gjorde vad? Nu kan vi veta!
- Minskade risker
  - Ha inte all kod på en enda dator
  - Låt inte vem som helst förändra koden

MALMÖ UNIVERSITET

Demo

Johan leker med Git

MALMÖ UNIVERSITET

Vad vill vi versionshantera?

Versionshanterar vi inte allt?

Nej. Viss data gör sig dåligt i ett repo.

I git hanterar vi detta genom att specificera vilka filer vi INTE vill hantera i filen .gitignore.

Ett problem som ni i era projekt måste lösa är hur ni hanterar känslig data och konfigurationsfiler. Tips: använd exempelkonfigurationsfiler och håll koll på känslig data utanför GitHub!

MALMÖ UNIVERSITET

Vad som versionshanteras

- Källkod, inklusive exempelvis tester
- Dokumentation
- Paket- och beroendeinformation, exempelvis:
  - *pom.xml* i Java-projekt
  - *gradle.yml* i Java- och Android-projekt
  - *package.json* i Javascript-projekt
  - *requirements.txt* i Python-projekt

MALMÖ UNIVERSITET

Vad som versionshanteras, del 2

- Viss metadata kring projekt:
  - Licensinformation
  - Git-relaterad data
    - .gitignore
    - .git

MALMÖ UNIVERSITET

Vad du INTE ska versionshantera!

- Känslig information
  - Användarnamn
  - Lösenord
  - API-nycklar
  - IP-adresser och URLer (127.0.0.1 och *example.com* är okej)
- Din IDE:s konfigurationsinformation
- Ditt operativsystems specialfiler, exempelvis *.DS_Store* på Mac
- Tillfälliga filer, exempelvis:
  - Data som laddas in med jämna mellanrum
  - Sessionsfiler

MALMÖ UNIVERSITET

Vad du INTE ska versionshantera #2

- Beroenden och bibliotek, exempelvis:
  - jar-filer i Java- och Android-projekt
  - dll-filer i Windows-projekt
  - *node_modules* i Javascript-projekt
  - *modules* i Python-projekt
- Binärfiler (om vi kan slippa det):
  - class-filer i Java- och Android-projekt
  - pyc-filer i Python-projekt
  - exe-filer i Windows-projekt

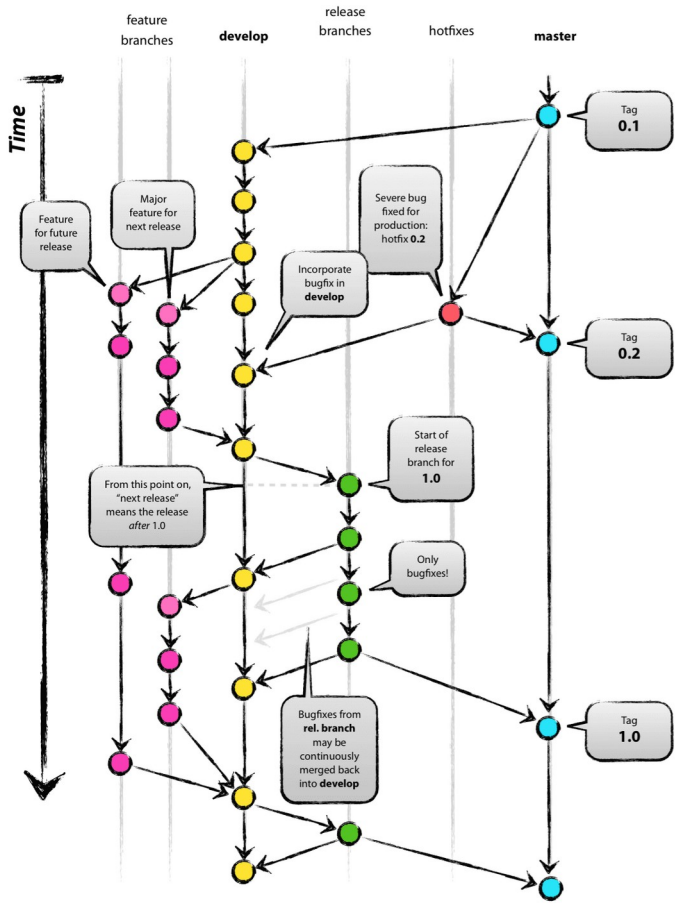
MALMÖ UNIVERSITET

Effektivt arbete med Git – Git Flow och Github Flow

Git Flow

En vanlig modell för att arbeta med Git

- Välanvänt och vältestat
- Använder mycket branching
- Väl anpassat för tighta utvecklingsteam
- Inte lika väl anpassat för större, löst organiserade team
  - Varje utvecklare måste kunna göra en push
- Negativt:
  - Långlivade grenar tenderar att leda till integrationsproblem
  - “Buskig” historik kan vara svår att följa

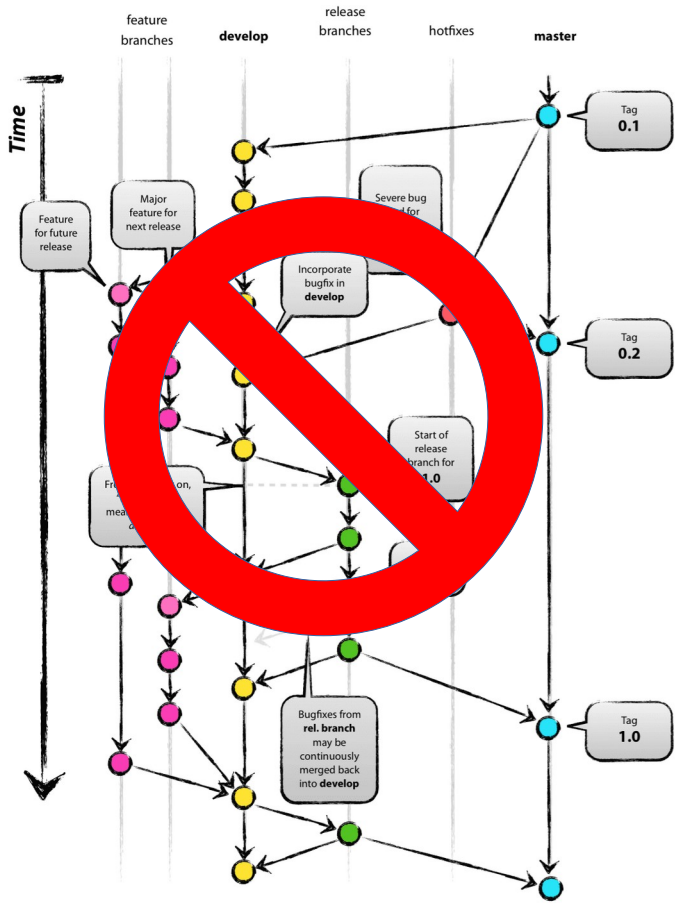


MALMÖ UNIVERSITY

GitHub Flow

En reaktion på Git Flow, avsett att

- Vara enklare att använda
- Mer avsett för CI/CD
- Negativt:
  - Master-branchen är inte garanterat körbar
  - Svår att arbeta med när du vill ha ett release-schema

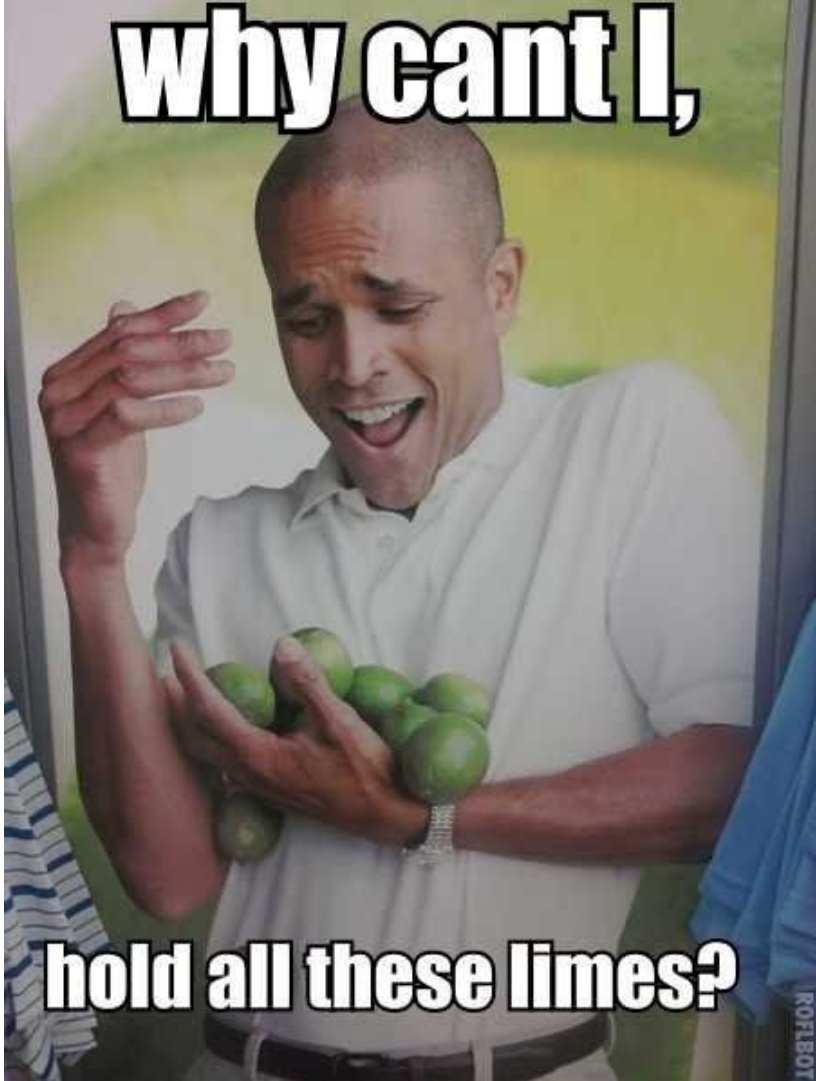


MALMÖ UNIVERSITY

Pakethantering

Vårt problem

- Riktiga mjukvaruprojekt är stora
- Riktiga mjukvaruprojekt innehåller oftast externt skriven mjukvara
- Hur hanterar vi detta på ett effektivt sätt?



Vad är pakethantering?

Modulbaserad mjukvara behöver kontrolleras – detta görs med fördel med en pakethanterare

- Återanvänd din (och andras) kod
- Separation of concerns
- Slipp ifrån jobbet med att dra ner paket manuellt

Usch!

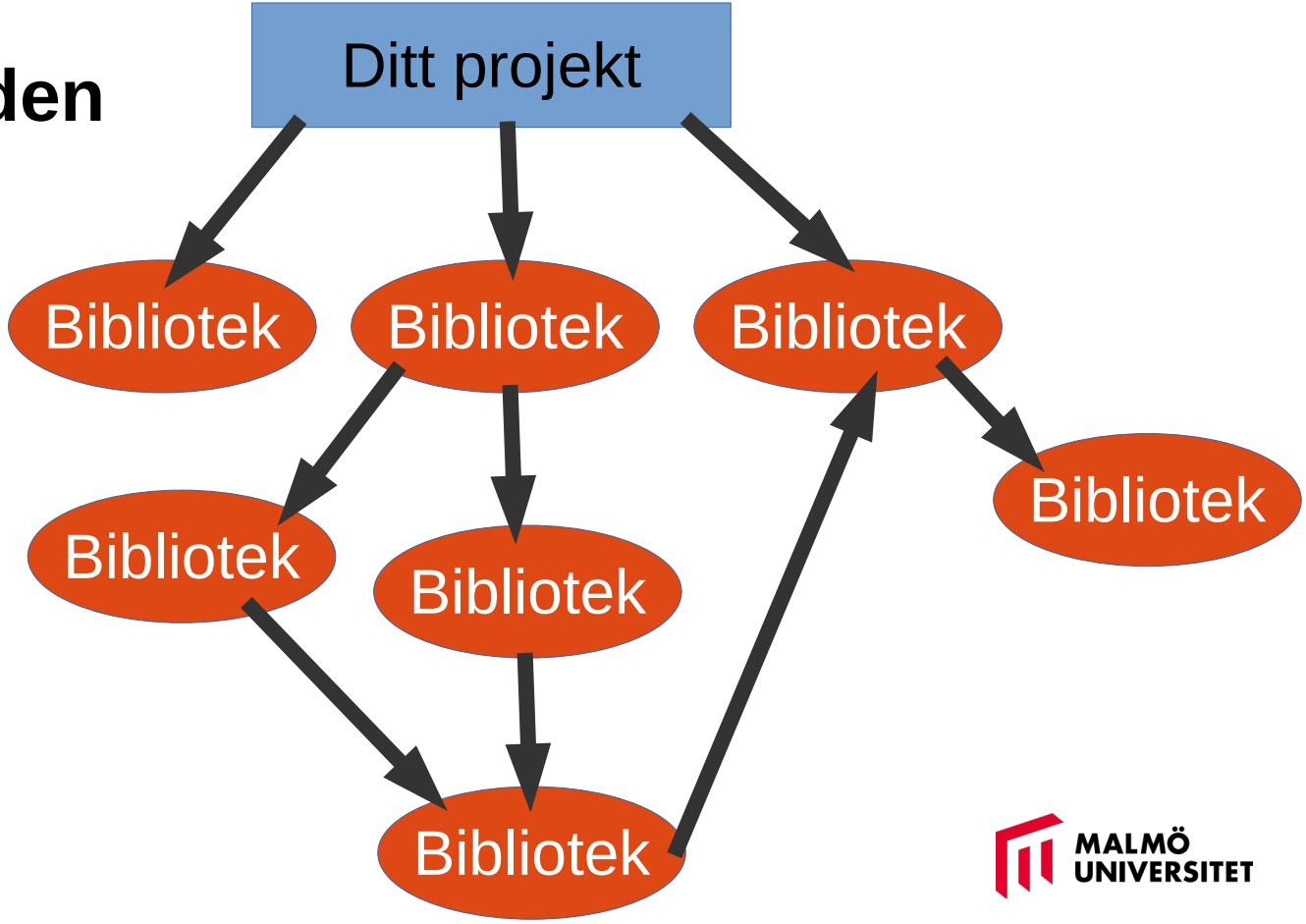
MALMÖ UNIVERSITET

Några viktiga termer

- **Paket**: En samling av filer som levereras tillsammans. Kan vara körbara filer, bibliotek eller andra resurser, så som media, typsnitt eller konfigurationsfiler. I JavaScript kallas dessa allmänt för **moduler**.
- **Pakethanterare**: En mjukvara som hanterar paket.
- **Repository** (eller **repo**): En plats där paket kan publiceras, sökas efter och laddas ner från.
- **Beroende**: Ett eller flera paket som behövs för att ett specifikt paket ska kunna användas.
- **Beroendekonflikt**: Ett tillstånd som uppstår då två paket beror på samma paket.
- **Version**: En specifik utgåva av ett specifikt paket. Genom att känna till versionsinformation kan en pakethanterare hålla koll på flera versioner av ett paket, vilket minskar risken för beroendekonflikter.

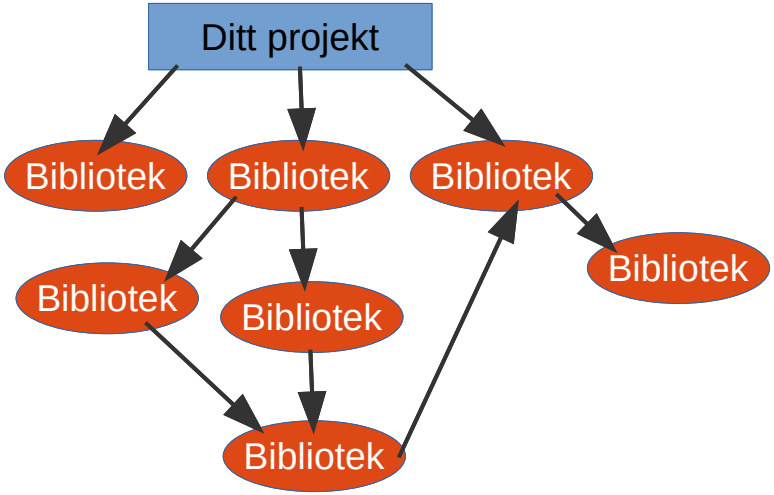
MALMÖ UNIVERSITET

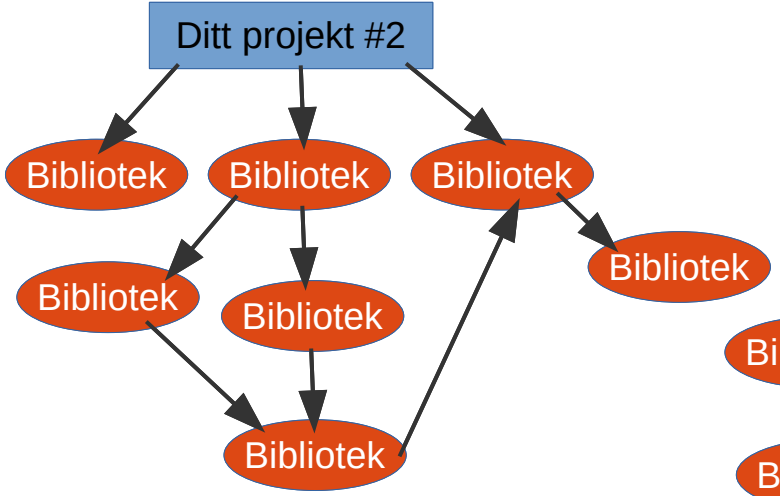
Beroenden

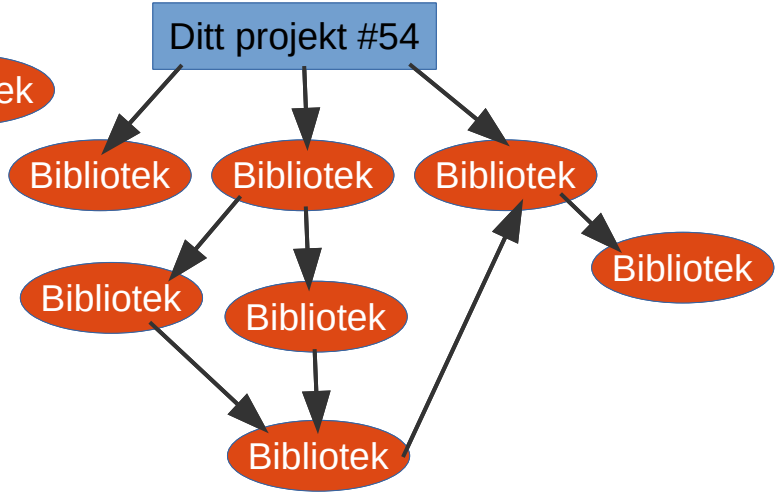


MALMÖ UNIVERSITET

Beroenden







MALMÖ UNIVERSITET

Beroendehantering i mjukvaruprojekt

- Python: pip
- Java: Maven/Gradle
- PHP: Composer
- C#/.NET: NuGet
- **Javascript: npm**

MALMÖ UNIVERSITET

Varför beroendehantera i mjukvaruprojekt?

- Robustare projekt och mjukvara
- Enklare att starta nya projekt
- Enklare för nya utvecklare att börja arbeta
- Mindre kod i era kod-repositories

MALMÖ UNIVERSITET

Pakethanteraren npm

- De facto-standard för vettig beroendehantering i JavaScript
  - ...men pakethanteraren Yarn vinner snabba insteg
- Bygger på Node.js – npm (Node Package Manager) är dess pakethanterare
  - Bygger på en kodstandard som heter CommonJS
  - Går att använda till annat än just Node.js
- Sköter installation av moduler åt dig
  - Du behöver inte längre versionshantera och leverera alla beroenden – bara en fil som berättar vilka paket som krävs

MALMÖ UNIVERSITET

Pakethanteraren npm

- Hanterar beroenden i flera led
- Kan skilja mellan olika versioner av moduler
- Förenklar uppdatering av mjukvaran när underliggande moduler uppdateras
- Dessutom kan npm användas för att automatisera paketering, testning och mycket annat!

Paketbeskrivning: package.json

```
{
  "name": "thin-red-line",
  "version": "0.5.7",
  "private": true,
  "scripts": {
    "start": "grunt docs && node ./bin/www",
    "test": "mocha --recursive test/unit_test -R dot",
    "system-test": "jasmine-node test/system_test"
  },
  "dependencies": {
    "bluebird": "~3.0.6",
    "body-parser": "~1.10.2",
    "xml2js": "^0.4.9"
  },
  "devDependencies": {
    "chai": "^3.0.0",
    "chai-as-promised": "5.1.*",
    "rewire": "~2.5.1"
  }
}
```

MALMÖ UNIVERSITET

Demo

Johan leker med npm

MALMÖ UNIVERSITET

npmjs.com

No Proscribed Meaning

npm Enterprise Products Solutions Resources Docs Support

npm Search packages Search

npm is the package manager for javascript

Popular libraries

- lodash
- request
- chalk
- react
- express
- commander
- moment
- debug
- async
- prop-types
- bluebird
- react-dom

Discover packages

- IoT
- mobile
- front end
- backend
- robotics
- css
- testing
- cli
- documentation
- math
- coverage
- frameworks

By the numbers

Packages

976 532

Downloads · Last Week

7 743 270 304

Downloads · Last Month

41 102 284 383

Recently updated packages

@controlla/controlla-postinstall

Lightweight npm postinstall message

`ivansotelo` published 1.0.19 • 3 minutes ago

@brd.com/deploy-it

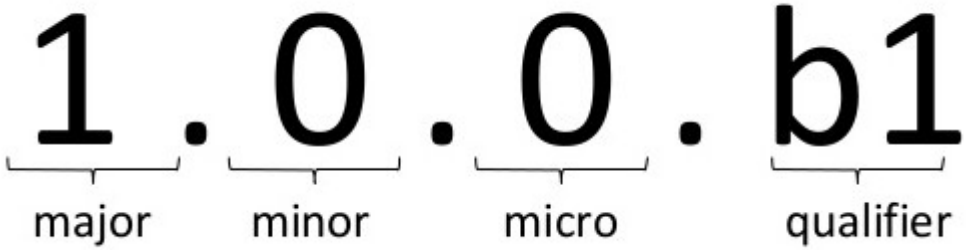
BRD's node deployment tools. In simple terms, this module has some sensible defaults for deploying plan node apps and nuxt apps. These get put together in the `gitlab-functions`, and can be overridden by javascript files in the `deploy` directory. Any

MALMÖ UNIVERSITET

npmjs.com

- Npm:s repository för JavaScript-moduler
- Modulerna är fria att använda i dina projekt
- Använder **semantisk versionering** för att skilja mellan olika versioner av moduler

Semantisk versionering



Semantisk versionering: http://semver.org/

MALMÖ UNIVERSITET

Alternativ till npm

- Yarn – I princip det enda alternativet till npm. Snabbare, kraftfullare, men inte lika välanvänt.

MALMÖ UNIVERSITET

Paketering

Vårt problem

- Riktiga mjukvaruprojekt är stora
- Riktiga mjukvaruprojekt behöver paketeras innan de levereras
- Hur hanterar vi detta på ett effektivt sätt?

Vad är paketering?

Utvecklad mjukvara innehåller som regel kod och artefakter som är ointressanta för slutanvändaren. Dessa tar stor plats och kan dessutom vara säkerhetshål. En paketerare tar bort alla onödiga artefakter och levererar en färdig mjukvara.

Begreppen “build” och “bundle”

- En build är den process som tar din utvecklingskod – källkod, grafiska element och annat – och slår samman dem till en enhet.
- En bundle är den artefakt som är resultatet av din build-process.

MALMÖ UNIVERSITET

Paketering med Webpack

- Webpack är ett av många – men ett av de mer användna – paketeringsverktygen för webbprojekt
- Skrivet i JavaScript
- Slår samman flera filer av samma typ till en (se demo)
  - Flera JavaScriptfiler blir en
  - Flera CSS-dokument blir ett, etc
- Genererar en uppsättning filer per HTML-dokument som ska användas
  - Sparar mycket laddningstid
- Klarar i grunden av JavaScript, men kan utökas med andra filtyper
- Hanteras med hjälp av konfigurationsfiler – precis som pakethanterarna

MALMÖ UNIVERSITET

Demo

Johan leker med Webpack

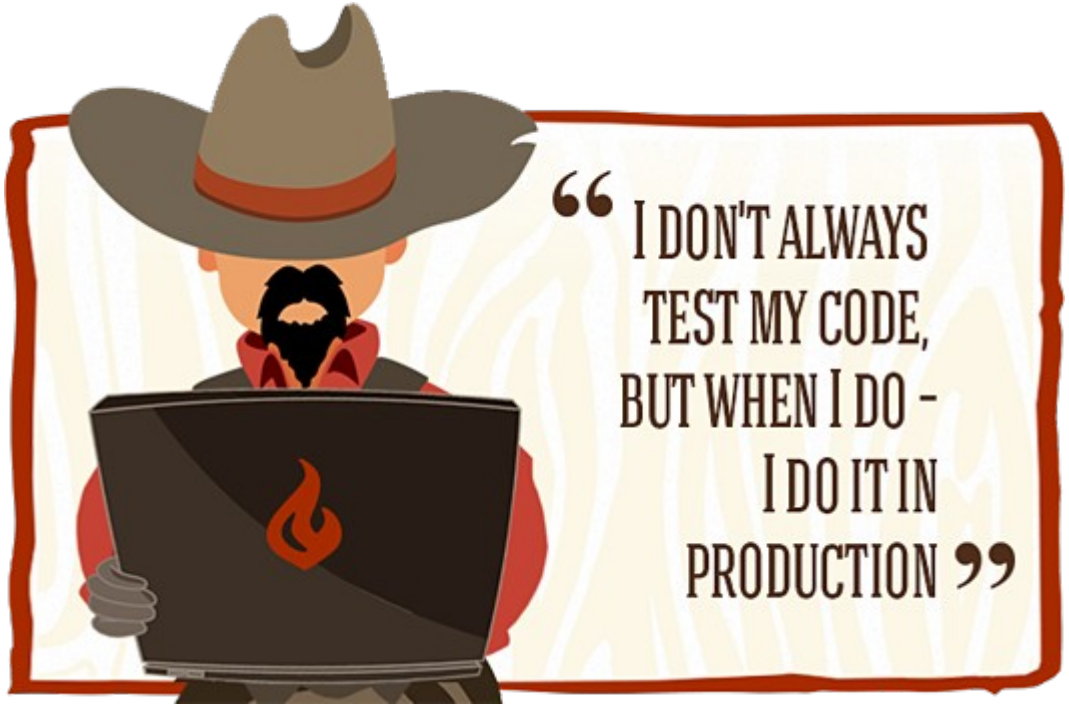
MALMÖ UNIVERSITET

Alternativ till Webpack

- Browserify – Ett kraftfullt verktyg som dock kräver hjälp av andra verktyg, som Gulp, för att utföra saker som att minimera script
- Parcel – Ett verktyg som gör en stor sak av att det inte kräver någon konfiguration
- Vite – Ett verktyg som inte heller kräver någon konfiguration och stödjer hot loading. Används av bland annat Vue (se nästa föreläsning)
- ...och så klart en massa andra

MALMÖ UNIVERSITET

Testning



MALMÖ UNIVERSITET

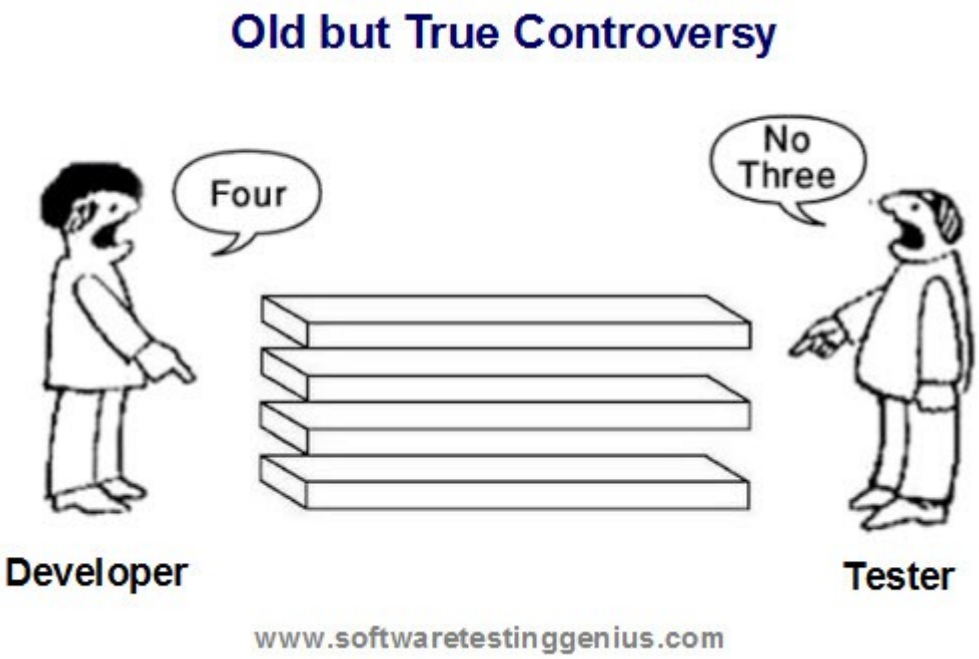
Vad är testning?

> **“Software testing** is an investigation conducted to provide stakeholders with information about the quality of the product or service under test.”
>
> Enl. Wikipedia

- Validering → Bygger vi rätt saker?
- Verifikation → Bygger vi sakerna på rätt sätt?

Wikipedia, *Software testing* – https://en.wikipedia.org/wiki/Software_testing

Varför testar vi våra applikationer?



MALMÖ UNIVERSITET

Varför testar vi våra applikationer?

- Tekniska skäl
- Utvecklingsteamets skäl
- Ekonomiska skäl

MALMÖ UNIVERSITET

Tekniska skäl: Säkerställ funktionaliteten

Det här är validering och verifiering!

- Fungerar koden som den är tänkt att göra?
  - Hanteras indatan på rätt sätt?
  - Fungerar koden med felaktig indata?
  - Är koden feltolerant?
- Kommer programmet att fungera i produktion?
  - Matchar vår utvecklingsmiljö produktionsmiljön?
  - Fungerar all kod tillsammans?

MALMÖ UNIVERSITET

My code working well on on my machine

** Deploys **

MALMÖ UNIVERSITET

Utvecklingsteamets skäl: Förtroende

- En bra utvecklare kan visa att hens kod fungerar
- Bra tester säkerställer att koden fungerar
  - Tester är bra att ha under utvecklingen av en funktion
  - De är ännu bättre att ha när koden har levt en tid
- Tester kan användas som bas i diskussioner

MALMÖ UNIVERSITET

Utvecklingsteamets skäl: Historik och nya utvecklare

Tester är dokumentation → förenklar introduktion av nya utvecklare

- Väl utformade och beskrivna tester visar hur en klass eller funktion ska fungera
- BDD-tester (user stories-baserade tester) beskriver hur applikationen ska fungera
- Lösta buggar och fel visas med tester

MALMÖ UNIVERSITET

Ekonomiska skäl:
Driftstörningar är dyrare än utvecklingstid

- Mjukvarutestning är dyrt
  - Längre utvecklingstid
  - Fler utvecklare/testare
  - Mer infrastruktur
- Fel i mjukvaran är dyrare
  - Nertid (se nästa slide)
  - Dålig PR/goodwill

MALMÖ UNIVERSITET

Ekonomiska skäl:
Driftstörningar är dyrare än utvecklingstid

Average Cost of Downtime

Even if you company survives a disaster, the costs are staggering:

- Brokerage $6M - $7M / hour
- Banking $5 - $6M / hour
- Credit Card $2M - $3M / hour
- Pay Per View $1 - $2M / hour (up to $50M for fights)
- Airline Reservations $1M / hour
- Home Shopping $100K / hour
- Catalog Sales $100K / hour
- Tele-ticket $70K / hour
- Package Shipping $30K / hour
- ATM Fees $20K / hour

Average mean time to repair or recover: 4.0 hours

Morpheus, *How to manage app uptime like a boss* - https://www.morpheusdata.com/blog/2016-04-06-how-to-manage-app-uptime-like-a-boss

MALMÖ UNIVERSITET

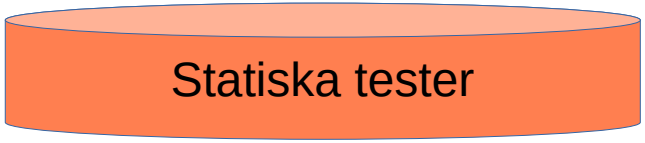
Olika typer av testning

- Statisk testning
- Enhetstestning
- Integrationstestning
- End-to-end/acceptanstest

MALMÖ UNIVERSITET

Statisk testning

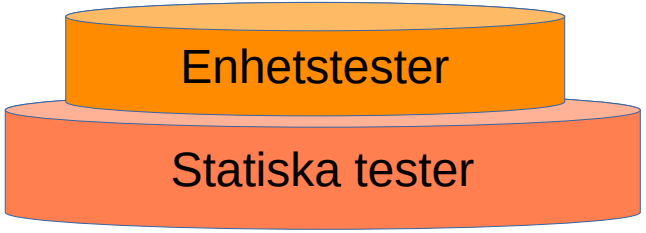
- Kräver ingen körning av mjukvaran
- Innefattar analys av:
  - Krav
  - Designdokument
  - Koden
- Kan göras både manuellt och med verktyg



MALMÖ UNIVERSITET

Enhetstestning

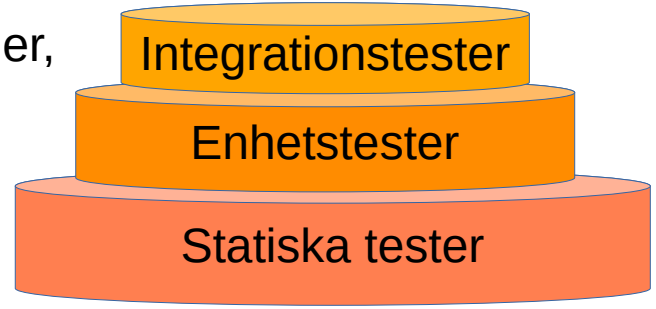
- Testning av små delar – enheter – av koden, exempelvis klasser eller funktioner
- Körs ofta – så fort koden ändras
- Testfallen bör definieras innan koden skrivs
- Utförs med verktyg, oftast automatiskt
- Kan ses som specifikation och dokumentation



MALMÖ UNIVERSITET

Integrationstestning

- Tester som involverar flera enheter av koden – exempelvis två moduler, klasser eller funktioner
- Görs för att säkerställa att enheterna fungerar tillsammans

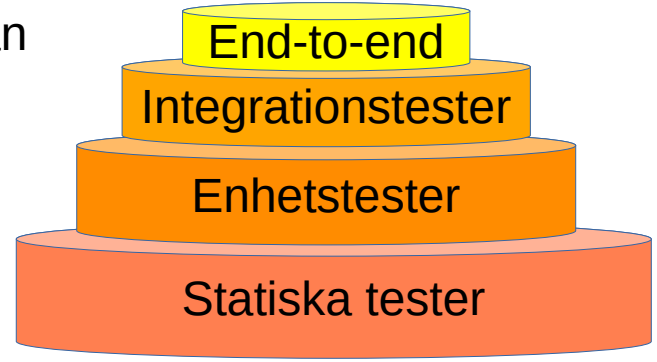


- Testerna körs ofta som en del av förberedelserna inför en release – projekt tenderar att använda specifika verktyg för detta

MALMÖ UNIVERSITET

End-to-end-testning

- Testar användarflöden i mjukvaran
- Heltäckande och testar många funktionaliteter samtidigt
- Körs inför leveranser
- Görs påfallande ofta av vanliga människor, men kan även göras med hjälp av automatiserade verktyg



MALMÖ UNIVERSITET

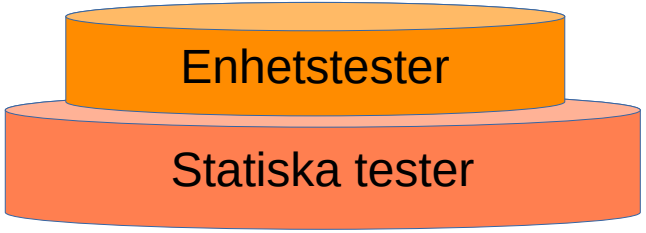
Testning i JavaScript/HTML

- Lintning för att säkerställa att vår kod och våra dokument är välformade:
  - ESLint eller JSLint för JavaScript
  - HTML Tidy för HTML
- Enhetstester
  - Jest
- End-to-end-tester
  - Selenium

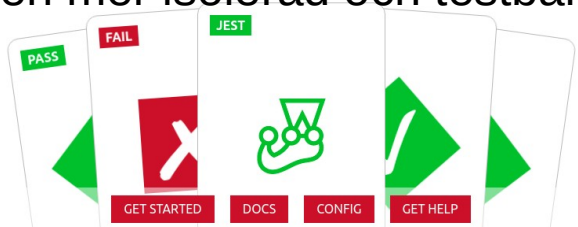
MALMÖ UNIVERSITET

Enhetstestning med Jest

- Bygger på samma tankesätt som ni sett i Junit (Java) och i unittest (Python)
- Testkoden lever separat från produktionskoden och kan köras direkt av npm



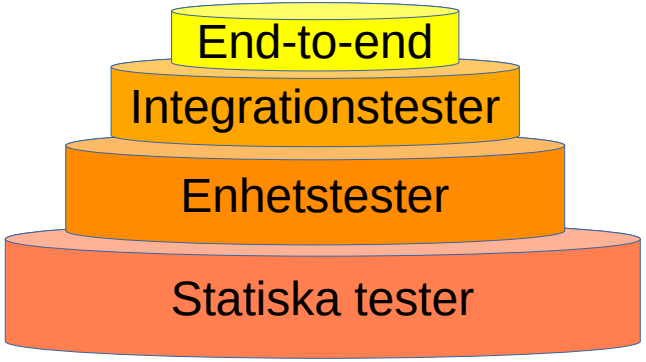
- Delar av koden kan ersättas med mock-objekt för att göra produktionskoden mer isolerad och testbar under testningen



MALMÖ UNIVERSITET

End-to-end-testning med Selenium

- Ett kraftfullt verktyg för att utföra frontend-tester i webbläsare
- Kan även användas för att automatisera arbetsflöden som kräver fysisk användning av webbläsaren



- Selenium körs med *web drivers*, som kan driva flera olika webbläsare under testningen
- Kan användas i flera olika språk och miljöer, men vi nöjer oss med att testa med JavaScript och Node

MALMÖ UNIVERSITET

Demo

Johan leker med Jest och Selenium

MALMÖ UNIVERSITET

Varför gör vi allt detta?

Ingen gör allt detta manuellt – vi döljer allt med olika ramverk som gör allt vi gått igenom + ännu mer

Det går Anton igenom på nästa föreläsning!

MALMÖ UNIVERSITET

Till sist

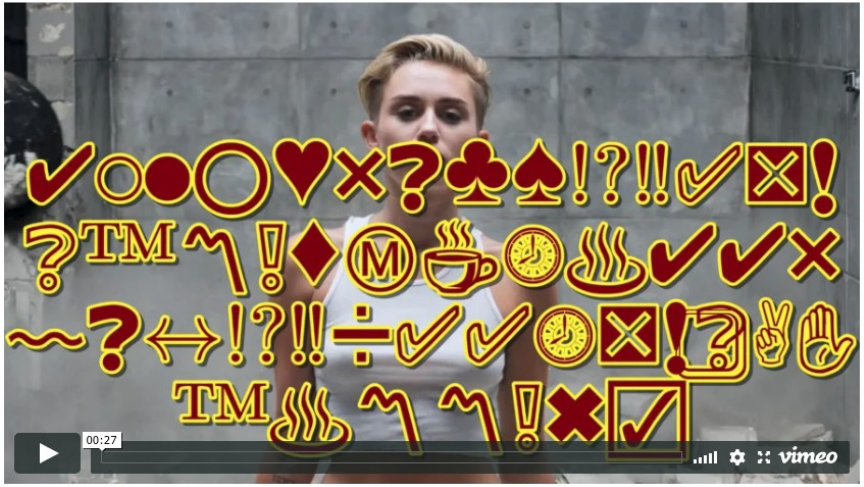
Glöm inte att programmering är kul!

Outcognito Mode

Outcognito Mode is a Chrome extension that publicly tweets every website you visit and everything you type. By Harold Cooper.

The Emoji Subtitle Creator

The Emoji Subtitle Creator by Ross Goodwin and Seth Kranzler automatically translates normal subtitles into ascii symbols.



MALMÖ UNIVERSITET

Det är okej att vara cowboy ibland

MALMÖ UNIVERSITET

What about projektet?

Projektinformation

- Anton har lagt upp information om projektet på Discord och webbsidan. Läs igenom den!

MALMÖ UNIVERSITET